Vault Enterprise on AWS EKS

Background

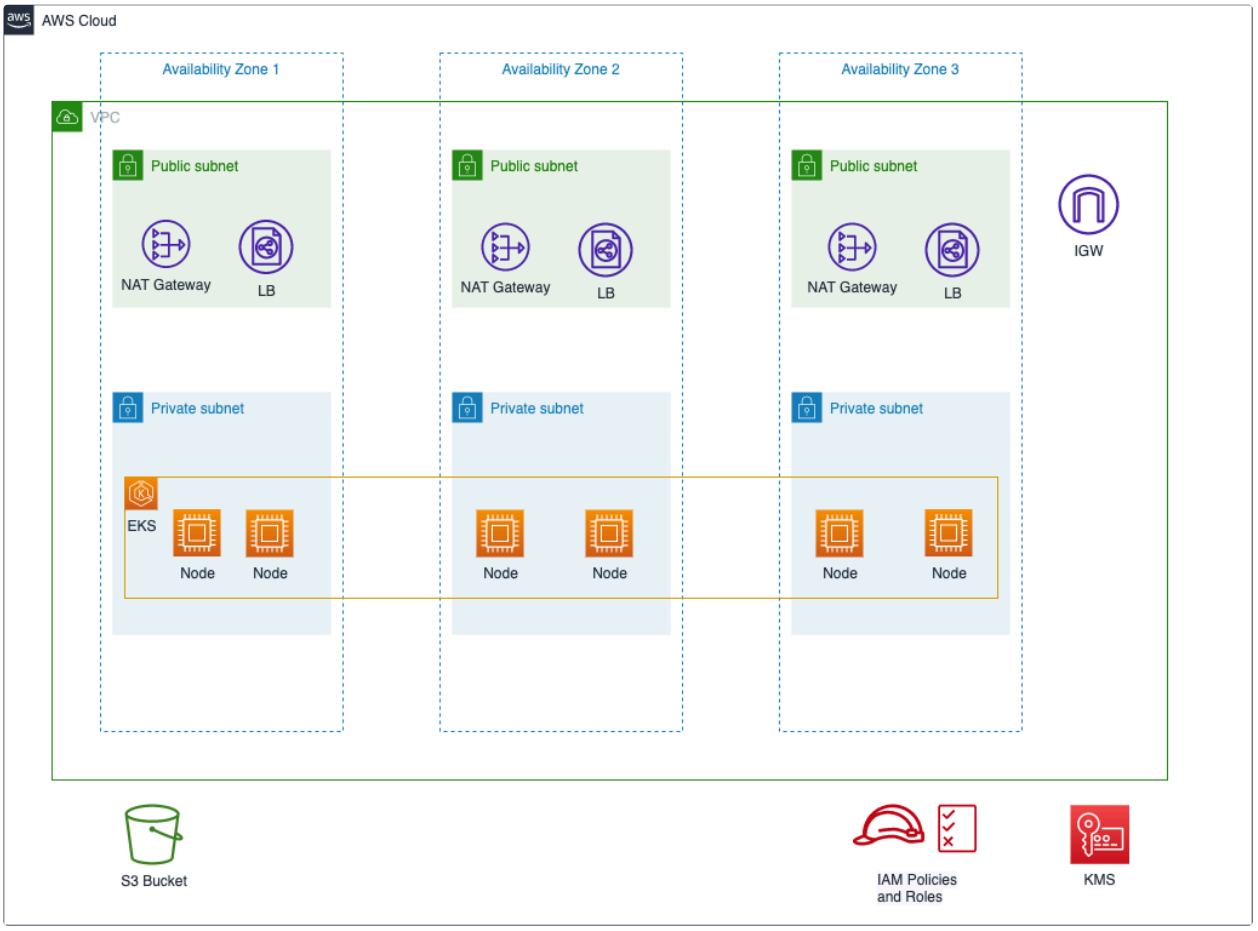
The parent guide described the generic architecture and deployment of Vault Enterprise on Kubernetes. It is a pre-requisite to understand how Vault is deployed on Kubernetes, the basic infrastructure and networking requirements and the kubernetes resources that gets created when the Vault Helm chart is run. However every cloud provider has their own managed Kubernetes platform and the details of architecture and deployment varies between them.

This guide specifically describes the architecture and deployment of Vault Enterprise on AWS Elastic Kubernetes Service (EKS) which is Amazon's managed k8s platform in AWS.

Solution Overview

Vault on EKS - Architecture and EKS Cluster Provisioning

Vault on Amazon EKS Reference Architecture



Vault on EKS - Infrastructure Architecture

The infrastructure required to be provisioned based on the Vault with IS RA tailored for k8s on AWS are

- AWS KMS key that will be used by Vault for auto-unseal.
- VPC with 3 public and 3 private subnets distributed over 3 AZs
- IGW and NATGW in the VPC
- EKS cluster with Node instance sizes as per spec from RA
  - Install the Amazon EBS CSI Driver addon - This is required for GP3 PVs.
  - Install the Amazon VPC CNI addon - This is required to enable pods getting IP address allocated from the VPC.
  - IAM permissions for above addons
  - AWS Load Balancer Controller for provisioning NLBs (the default creates classic ELBs)
  - IAM permission for the LB Controller
- NLB for load balancing - Note: This is automatically created from k8s services by the cloud controller
- KMS key for Vault auto-unseal
- SSH key pair to configure SSH access on k8s nodes.
- S3 bucket for snapshot storage

Create KMS key

This key will be used by Vault for auto-unseal.

```
$ KEY_ID=$(aws kms create-key  --output json | jq -r .KeyMetadata.KeyId)
$ echo $KEY_ID
$ aws kms create-alias --alias-name "alias/vault-pri-unseal" --target-key-id $KEY_ID
```

Create IAM Policy required for KMS auto-unseal

Create a customer managed policy file named `vault-pri-kms-seal-policy.json`. Use the correct Region, AccountId and KeyId.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "kms:Decrypt",
                "kms:Encrypt",
                "kms:DescribeKey"
            ],
            "Resource": "arn:aws:kms:ap-southeast-2:617888641103:key/$KEY_ID"
        }
    ]
}
```

```
$ aws iam create-policy \
    --policy-name vault-pri-kms-seal-policy \
    --policy-document file://vault-pri-kms-seal-policy.json
```

It is best practice to use different KMS keys to unseal each cluster, as using separate KMS key for each cluster reduces the blast radius in the event that a KMS key was lost.

Create IAM Policy required for AWS Load Balancer Controller

```
$ curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.4.7/docs/install/iam_policy.json

$ aws iam create-policy \
    --policy-name AWSLoadBalancerControllerIAMPolicy \
    --policy-document file://iam_policy.json
```

Provision EKS cluster

Install eksctl

`eksctl` is a simple CLI tool for creating and managing clusters on EKS. It abstracts away the complexity of provisioning EKS clusters but of course Terraform can also be used instead.

```
$ brew upgrade eksctl && { brew link --overwrite eksctl; } || { brew tap weaveworks/tap; brew install weaveworks/tap/eksctl; }
$ eksctl version
$ mkdir -p ~/.zsh/completion/
$ eksctl completion zsh > ~/.zsh/completion/_eksctl
$ source ~/.zshrc
```

Create the cluster configuration file

Create a `vault-pri-cluster.yaml` file that describes the configuration for the primary Vault EKS cluster.

> Configuration values such AccountId, region, VPC CIDR etc. as should be changed as required.

The various sections of the configuration and the values are explained below.

Section	Description

`metadata:`
`  name: vault-pri`
`  region: ap-southeast-2`
`  tags: { donotdelete: "true", owner: "pratik" }`
	The Name of the EKS Cluster, region and tags

`vpc:`
`  cidr: 10.12.0.0/16`
`  nat:`
`  gateway: Single #HighlyAvailable`
	The VPC CIDR and NAT Gateway type.
	
	Recommended to be set to HighlyAvailable for Production deployment.

`managedNodeGroups:`
`  - name: ng-vault`
`  labels: { role: workers }`
`  instanceType: m5.large`
`  minSize: 3`
	The configuration of the Node Group.

```
    desiredCapacity: 3
    maxSize: 6
    volumeSize: 80
```

Min and Desired size should be set as per RA for Prod and Non-Prod environment.

The InstanaceType should be set as per RA for Prod and Non-Prod environment.

> Vault HELM chart currently doesn't support Autp-Pilot redundancy zones. When that is supported the MIn and Desired size for Prod should be set to 6

```
cloudWatch:
  clusterLogging:
    enableTypes: ["*"]
```

Enabling all categories of cluster logging to CloudWatch

```
iam:
  withOIDC: true

```

This creates an IAM OIDC provider for the cluster.

This is required for IAM roles for service accounts - Amazon EKS (IRSA) to work.

```
serviceAccounts:
    - metadata:
        name: vault
        namespace: vault
      attachPolicyARNs:
        - "arn:aws:iam::617888641103:policy/vault-kms-seal-policy"
      roleOnly: true
      roleName: eksctl-vault-pri-kms-seal-role

```

This will create a IAM role with the `vault-kms-seal-policy` policy attached with trust relationship to a k8s Service Account named `vault` in the namespace `vault`

This is required by Vault to use KMS for auto-unseal. Using IRSA the permission is only provided to the vault pods which runs with the identity of the `vault` Service Account in the `vault` namespace. Providing the IAM role at Node level would have given the permission to all pods including any non-vault pods running on the Node which is against the principle of least privilege.

> Note: only the IAM Role is created not the SA. The vault SA is created later

	during the HELM chat installation

``yaml
- metadata:
  name: ebs-csi-controller-sa
  namespace: kube-system
  attachPolicyARNs:
  - "arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy"
  roleOnly: true
  roleName: eksctl-vault-pri-AmazonEBSCSIDriverRole
``
 	
This will create a IAM role with the `AmazonEBSCSIDriverPolicy` policy attached with trust relationship to a k8s Service Account named `ebs-csi-controller-sa` in the namespace `kube-system`
 	
This Role has permissions required by the EBS CSI Driver to manage storage volumes.
 	
Note: only the IAM Role is created not the SA. The SA is created later when the Amazon EBS CSI Driver addon is installed later.	

``yaml
- metadata:
  name: aws-load-balancer-controller
  namespace: kube-system
  attachPolicyARNs:
  - "arn:aws:iam::617888641103:policy/AWSLoadBalancerControllerIAMPolicy"
  roleName: eksctl-vault-pri-AWSLoadBalancerControllerRole
``
 	
This will create a IAM role with the `AWSLoadBalancerControllerIAMPolicy` policy attached with trust relationship to a k8s Service Account named `aws-load-balancer-controller` in the namespace `kube-system`
 	
This Role has permissions required by the AWS Load Balancer controller.	

Eksctl - Config File Schema

The full `vault-pri-cluster.yaml` file is shown below:

```yaml
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: vault-pri
  region: ap-southeast-2
  tags: { donotdelete: "true", owner: "pratik" }

vpc:
  cidr: 10.12.0.0/16
  nat:
    gateway: Single #HighlyAvailable

```

```yaml
managedNodeGroups:
  - name: ng-vault
    labels: { role: workers }
    instanceType: m5.large
    minSize: 3
    desiredCapacity: 3
    maxSize: 6
    volumeSize: 80

cloudWatch:
  clusterLogging:
    enableTypes: ["*"]

iam:
  withOIDC: true
  serviceAccounts:
    - metadata:
        name: vault
        namespace: vault
      attachPolicyARNs:
        - "arn:aws:iam::617888641103:policy/vault-pri-kms-seal-policy"
      roleOnly: true
      roleName: eksctl-vault-pri-kms-seal-role
    - metadata:
        name: ebs-csi-controller-sa
        namespace: kube-system
      attachPolicyARNs:
        - "arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy"
      roleOnly: true
      roleName: eksctl-vault-pri-AmazonEBSCSIDriverRole
    - metadata:
        name: aws-load-balancer-controller
        namespace: kube-system
      attachPolicyARNs:
        - "arn:aws:iam::617888641103:policy/AWSLoadBalancerControllerIAMPolicy"
      roleName: eksctl-vault-pri-AWSLoadBalancerControllerRole
```

Provision EKS cluster

```bash
$ eksctl create cluster -f vault-pri-cluster.yaml --dry-run
$ eksctl create cluster -f vault-pri-cluster.yaml

$ kubectl config get-contexts
$ kubectl config rename-context pratik@hashicorp.com@vault-pri.ap-southeast-2.eksctl.io vault-pri

$ kubectl get pods -A
$ kubectl get sa -A

$ aws iam list-roles | jq '.Roles[].RoleName' | grep eksctl-vault-pri
$ aws iam get-role --role-name eksctl-vault-pri-kms-seal-role | jq -r '.Role.Arn'
$ aws iam get-role --role-name eksctl-vault-pri-AmazonEBSCSIDriverRole | jq -r '.Role.Arn'
$ aws iam get-role --role-name eksctl-vault-pri-AWSLoadBalancerControllerRole | jq -r '.Role.Arn'
```

IAM Role for Service Accounts (IRSA)

IRSA is a way to assign IAM permission to k8s Service Account to restrict access to AWS services.

IRSA provides the benefits of

- Least Privilege
- Credential Isolation

- Auditability

As the IAM role within the EC2 Instance Profile of the Node instances does not have necessary permissions to encrypt/decrypt with KMS key, Vault will get an “Access Denied” error during auto-unseal process. One way to fix this could be to attach additional permissions to the EC2 instance profile. However, this violates a key security principle, the principle of least privilege. This additional permission would be at the EC2 Node level, not at the Kubernetes Pod level. Therefore, all Pods running on that node would gain access to the KMS key. We want to restrict this permission to the Pod level for only Vault pods.

The Role will have a trust relationship as follows:

`4A75D787B498A9546327F3FBF9218C55` is the OIDC Id for the EKS cluster. The `sub` claim is the specific k8s Service Account for vault.

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Principal": {
            "Federated": "arn:aws:iam::617888641103:oidc-provider/oidc.eks.ap-southeast-2.amazonaws.com/id/FB7AF27AF01C5557DF604442460EAF43"
         },
         "Action": "sts:AssumeRoleWithWebIdentity",
         "Condition": {
            "StringEquals": {
               "oidc.eks.ap-southeast-2.amazonaws.com/id/FB7AF27AF01C5557DF604442460EAF43:aud": "sts.amazonaws.com",
               "oidc.eks.ap-southeast-2.amazonaws.com/id/FB7AF27AF01C5557DF604442460EAF43:sub": "system:serviceaccount:vault:vault"
            }
         }
      }
   ]
}
```

Create the addons configuration file

The following addons are required to be installed on the EKS cluster.

- aws-ebs-csi-driver: A Kubernetes Container Storage Interface (CSI) plugin that provides Amazon EBS storage for your cluster. This is required to create the gp3 Storage Class for Vault data storage as per RA.
- vpc-cni: A Kubernetes Container Network Interface (CNI) plugin that provides native VPC networking for your cluster. This addon is required for NLB targeting Pod IPs instead of Node instances. The plugin allows pods to have IP address from the VPC.

Each addon requires specific IAM permissions which are provided through the IAM Roles that have already been provisioned before.

Create a `vault-pri-cluster-addons.yaml` file that describes the configuration for the addons.

> Note: Check and use the latest versions of the addons before provisioning.

```yaml
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: vault-pri
  region: ap-southeast-2
  version: "1.24"
  tags: { donotdelete: "true", owner: "pratik" }

addons:
  - name: vpc-cni
    version: v1.12.6-eksbuild.1
    attachPolicyARNs:
      - arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy
    resolveConflicts: overwrite
  - name: aws-ebs-csi-driver
    version: v1.17.0-eksbuild.1
    serviceAccountRoleARN: arn:aws:iam::617888641103:role/eksctl-vault-pri-AmazonEBSCSIDriverRole
    resolveConflicts: overwrite
```

Install required addons

```
$ eksctl create addon -f vault-pri-cluster-addons.yaml

$ eksctl get addons --cluster vault-pri
```

Install AWS Load Balancer Controller

The AWS Load Balancer Controller manages AWS Elastic Load Balancers for a Kubernetes cluster. This controller is required to provision NLBs for vault LoadBalancer services with pod IP targets as per RA. The default cloud controller provisions classic ELBs which are deprecated and cannot provide IP level targeting and other advanced capabilities. NLB is the recommended Layer 4 LB that should be used for Vault.

📦Route internet traffic with AWS Load Balancer Controller - Amazon EKS

```
$ helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=vault-pri \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller

$ kubectl get deployment -n kube-system aws-load-balancer-controller
```

Deploy the Storage Class for gp3

Vault with IS backend requires fast SSD disks and as per RA gp3 volumes should be used for vault data and audit storage. The new gp3 volumes are designed to provide a predictable, baseline performance of 3,000 IOPS regardless of the volume size. The default persistent volumes (PV) that are created in EKS is gp2. We need to create the Storage Class for gp3 using the EBS CSI driver plugin that has been previously installed.

Create a config file `gp3storageclass.yaml`

```yaml
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: gp3
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
reclaimPolicy: Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

Deploy the Storage Class.

```
$ kubectl apply -f gp3storageclass.yaml
$ kubectl get sc
```

Summary

At this point we have deployed an EKS cluster with configurations and addons to conform to the Reference Architecture.

The next step is to deploy Vault Enterprise on this EKS cluster.

Vault on EKS - Deployment

Vault on EKS Deployment

The [Vault Helm chart]() is the recommended way to install and configure Vault on Kubernetes. In addition to running Vault itself, the Helm chart is the primary method for installing and configuring Vault to integrate with other services such as Consul for High Availability (HA) deployments.

> ⚠ By default, the chart deploys a standalone vault. This mode uses a single Vault server with a file storage backend. This does not conform to Vault Ent HA RA.

The steps and configuration values to deploy a production grade Vault Enterprise cluster on a EKS K8S cluster is documented below. This example deploys a Vault Primary cluster.

Create Kubernetes Namespace

```
$ kubectl create namespace vault
```

Create Certificates for Vault

This guide uses openssl generated self-signed Root CA cert and server cert.

```
./create-ca.sh "platformrengineering root"
./create-cert.sh vault.platformrengineering.au
```

Create required k8s secrets

Create a k8s secret to store the CA Root cert public key

```
$ kubectl --namespace vault create secret generic vault-ca-crt --from-file=ca.crt
```

Create a k8s secret to store the Vault server cert public and private key

```
$ kubectl --namespace vault create secret tls tls-server --cert=server.crt --key=server.key
```

Create a k8s secret to store the Vault Enterprise license file

```
$ kubectl --namespace vault create secret generic vault-ent-license --from-file=license=./vault_license.hclic
$ kubectl --namespace vault get secrets
```

> These secrets are mounted as files in the Vault pods in the path `/vault/userconfig/<secret-name>/<secret-key>`

Setup Helm Repo

```
$ helm repo add hashicorp https://helm.releases.hashicorp.com
$ helm search repo hashicorp/vault
```

Create configuration file for Helm chart

Create a `override-values.yaml` file to specify the values to override the default values in the chart.

The sections and values used in the file are described in the table.

Section	Description

`server:`
`  # Use the Enterprise Image`
`  image:`
`  repository: "hashicorp/vault-enterprise"`
`  tag: "1.12.3-ent"`
`  enterpriseLicense:`
`  secretName: vault-ent-license`
	Set the Vault Ent version
	
	Set the k8s secret in which the license file is stored

`  readinessProbe:`
`  enabled: true`
`  path: "/v1/sys/health?perfstandbyok=true&drsecondarycode=200&sealedcode=204&uninitcode=204"`
`  initialDelaySeconds: 60`
	Set the readinessProbe
	
	Set the livenessProbe

``yaml
  livenessProbe:
    enabled: true
    path: "/v1/sys/health?perfstandbyok=true&drsecondarycode=200"
    initialDelaySeconds: 60
``
``yaml
extraEnvironmentVars:
    VAULT_CACERT: /vault/userconfig/vault-ca-crt/ca.crt
    AWS_REGION: "ap-southeast-2"
    VAULT_AWSKMS_SEAL_KEY_ID: "c56a9429-76cf-4f97-ab89-f2cf4e1cbb2f"
``
``yaml
extraVolumes:
    - type: secret
      name: tls-server
    - type: secret
      name: vault-ca-crt
``
``yaml
dataStorage:
    size: 100Gi
    storageClass: gp3
``
``yaml
auditStorage:
    enabled: false
    storageClass: gp3
``
``yaml
standalone:
    enabled: false
``
``yaml
ha:
    enabled: true
    replicas: 5
    raft:
      enabled: true
      setNodeId: true
      performance_multiplier: 1

	recommended for production Vault servers.

``
listener "tcp" {
tls_disable = false
tls_disable_client_certs = true
address = "[::]:8200"
cluster_address = "[::]:8201"
# Enable unauthenticated metrics access 	
telemetry {
unauthenticated_metrics_access = "true"
}
tls_cert_file = "/vault/userconfig/tls-server/tls.crt"
tls_key_file = "/vault/userconfig/tls-server/tls.key"
}
``
 	
Configures Vault to listen on a TCP address/port.
• `tls_disable = false` : TLS is enabled on the listener. Vault will only have HTTPS endpoint for API.
• `tls_disable_client_certs = true` : Turns off client authentication for this listener. This prevents a browser prompt to select a client certificate when browsing the Vault UI.
• `address` : The address to bind to for listening.
• `cluster_address` : The address to bind to for cluster server-to-server requests
• `unauthenticated_metrics_access = "true"` : Allows unauthenticated access to the `/v1/sys/metrics` endpoint. Necessary for Prometheus Operator
• `tls_cert_file` : The file for the TLS server cert public key
• `tls_key_file` : The file for the TLS server cert private key
ℹ Note: The certs file paths are specified to the path in which the k8s secrets holding the certs are mounted	

``
storage "raft" {
path = "/vault/data"
retry_join {
auto_join = "provider=k8s namespace={{.Release.Namespace}} label_selector=\"app.kubernetes.io/name={{ template "vault.name" . }},component=server\""
leader_ca_cert_file = "/vault/userconfig/vault-ca-crt/ca.crt"
``
 	
Configures the Raft (Integrated Storage) storage backend
• `path = "/vault/data"` : This is the path to store the Vault data. The path is mounted on the dataStorage PVs specified in the Helm chart	

```
    leader_client_cert_file = "/vault/userconfig/tls-server/tls.crt"
    leader_client_key_file = "/vault/userconfig/tls-server/tls.key"
    leader_tls_servername = "*.vault-internal"
  }


  autopilot {
    cleanup_dead_servers = "true"
    last_contact_threshold = "200ms"
    last_contact_failure_threshold = "10m"
    max_trailing_logs = 250000
    min_quorum = 3
    server_stabilization_time = "10s"
  }
}
```

- `auto_join = "provider=k8s namespace= {{.Release.Namespace}} label_selector=\"app.kubernetes.io/name={{ template "vault.name" . }},component=server\""` : This configures auto join using the k8s provider. Vault uses the [go-discover](#) library to automatically discover the other Vault pods in the k8s cluster using the label_selector specified and joins them in a raft quorum.
- The various certificate file paths for possible leader node are given
- `leader_tls_servername` : The TLS server name to use when connecting with HTTPS. Should match one of the names in the DNS SANs of the server certificate. Since each vault pod gets a unique server name in the statefulset (vault-0.vault-internal etc.) a wildcard name is specified. This has already been added to the SAN when issuing the server cert.
- The default autopilot settings are given.

```
seal "awskms" {
}
```

Sets the seal type as AWS KMS. see https://developer.hashicorp.com/vault/tutorials/day-one-raft/autounseal-aws-kms

No other config values needs to be specified.

- The region and KMS key id are specified through env vars in the Helm chart.
- The authentication to KMS is done via the IAM Roles for Service

Account configured for the `vault` k8s service account for the Vault pods.

```
cluster_name = "vault_pri"
log_level = "trace"
max_lease_ttl = "768h"
default_lease_ttl = "24h"
service_registration "kubernetes" {}
api_addr     = "https://HOSTNAME.vault-internal:8200"
cluster_addr = "https://HOSTNAME.vault-internal:8201"
ui = true
```

Sets some other values for the Vault cluster

- `cluster_name` : Friendly name for the Vault cluster
- `log_level` : Sets logging level to `trace`
- `max_lease_ttl` : Sets Max Lease TTL to 32 days
- `default_lease_ttl` : Sets Default Lease TTL to 1 days. **It is recommended to set the default TTL to a short value.**
- `service_registration "kubernetes" {}` : Kubernetes Service Registration tags Vault pods with their current status for use with selectors. see Kubernetes service registration | Vault | HashiCorp Developer
- `api_addr` : The address and port to be used for Vault API. Set to the hostname of the Vault pod from the headless vault-internal service. e.g. `https://vault-0.vault-internal:8200`
- `cluster_addr` : The address and port to be used for communication between the nodes in the Raft cluster. Set to the hostname of the Vault pod from the headless vault-internal service. e.g. `https://vault-0.vault-internal:8201`
- `ui = true` : Enable Vault UI

```
serviceAccount:
```

Sets the IAM Role for Service Account

```
    annotations:
      {
        eks.amazonaws.com/role-arn: "arn:aws:iam::617888641103:role/eksctl-vault-pri-kms-seal-role",
      }
```

The chart creates a SA named `vault`. The IAM Role ARN for the SA is specified through annotations. The Role was created when the EKS cluster was provisioned earlier.

```
service:
    type: LoadBalancer
    externalTrafficPolicy: Local
    annotations:
      {
        service.beta.kubernetes.io/aws-load-balancer-type: external,
        service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing,
        service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
      }
  ingress:
    enabled: false
```

Configures the k8s services created.

The chart creates several k8s services as has been described in a previous section. The services default configuration is overriden here.

- `type` : The service type is `LoadBalancer`. The AWS Load Balancer controller will provision Layer 4 NLB to expose the services externally. This conforms to RA.
- `externalTrafficPolicy: Local` : The client IP addresses is propagated to the end Pods by the NLB instead of the IP address of the node. This is useful as it will log client IPs in audit log, CIDR restrictions on token can be enabled etc.
- Annotations
  - `aws-load-balancer-type: external` :
  - `aws-load-balancer-scheme: internet-facing` :
  - `aws-load-balancer-nlb-target-type: ip` : AWS NLB sends traffic directly to the Vault pods behind the service, eliminating the need for an extra network hop through the worker nodes in the cluster.
- `ingress:  enabled: false` : Disables ingress. Ingress creates Layer

	7 LB and is not recommended for Vault.

``yaml
ui:
  enabled: true
  serviceType: "LoadBalancer"
  publishNotReadyAddresses: false
  activeVaultPodOnly: true
``
Configures the Vault UI service.
• `enabled: true` : Enables the service for Vault UI
• `serviceType: "LoadBalancer"`
• `publishNotReadyAddresses: false` : Wait for pods to be ready
• `activeVaultPodOnly: true` : Only contain selectors for active Vault pod. i.e. The UI service backend is the active Vault pod only.	

GitHub - hashicorp/vault-helm: Helm chart to install Vault and other associated components.

The full `override-values.yaml` file is shown below:

```yaml
# Vault Helm Chart Value Overrides
global:
  enabled: true
  tlsDisable: false

server:
  # Use the Enterprise Image
  image:
    repository: "hashicorp/vault-enterprise"
    tag: "1.12.3-ent"
  enterpriseLicense:
    secretName: vault-ent-license

  # For HA configuration and because we need to manually init the vault,
  # we need to define custom readiness/liveness Probe settings
  readinessProbe:
    enabled: true
    path: "/v1/sys/health?perfstandbyok=true&drsecondarycode=200&sealedcode=204&uninitcode=204"
    initialDelaySeconds: 60
  livenessProbe:
    enabled: true
    path: "/v1/sys/health?perfstandbyok=true&drsecondarycode=200"
    initialDelaySeconds: 60

  # extraEnvironmentVars is a list of extra environment variables to set with the stateful set. These could be
  # used to include variables required for auto-unseal.
  extraEnvironmentVars:
    VAULT_CACERT: /vault/userconfig/vault-ca-crt/ca.crt
    AWS_REGION: "ap-southeast-2"
    VAULT_AWSKMS_SEAL_KEY_ID: "c56a9429-76cf-4f97-ab89-f2cf4e1cbb2f"
    # AWS_REGION: "ap-southeast-1"
    # VAULT_AWSKMS_SEAL_KEY_ID: "24691abb-7085-445c-9d9b-f97030d0f512"

  # extraVolumes is a list of extra volumes to mount. These will be exposed
```

```
  # to Vault in the path `/vault/userconfig/<name>/`.
  extraVolumes:
    - type: secret
      name: tls-server
    - type: secret
      name: vault-ca-crt

  dataStorage:
    size: 100Gi
    storageClass: gp3

  # This configures the Vault Statefulset to create a PVC for audit logs.
  # See https://www.vaultproject.io/docs/audit/index.html to know more
  auditStorage:
    enabled: false
    storageClass: gp3

  standalone:
    enabled: false

  # Run Vault in "HA" mode.
  ha:
    enabled: true
    replicas: 3
    raft:
      enabled: true
      setNodeId: true
      performance_multiplier: 1

      config: |
        listener "tcp" {
          tls_disable = false
          tls_disable_client_certs = true
          address = "[::]:8200"
          cluster_address = "[::]:8201"
          # Enable unauthenticated metrics access (necessary for Prometheus Operator)
          telemetry {
            unauthenticated_metrics_access = "true"
          }
          tls_cert_file = "/vault/userconfig/tls-server/tls.crt"
          tls_key_file = "/vault/userconfig/tls-server/tls.key"
        }

        storage "raft" {
          path = "/vault/data"
            retry_join {
            auto_join = "provider=k8s namespace={{.Release.Namespace}} label_selector=\"app.kubernetes.io/name={{ template "vault.name" . }},component=server\""
            leader_ca_cert_file = "/vault/userconfig/vault-ca-crt/ca.crt"
            leader_client_cert_file = "/vault/userconfig/tls-server/tls.crt"
            leader_client_key_file = "/vault/userconfig/tls-server/tls.key"
            leader_tls_servername = "*.vault-internal"
            tls_disable_client_certs = "true"
          }


          autopilot {
            cleanup_dead_servers = "true"
            last_contact_threshold = "200ms"
            last_contact_failure_threshold = "10m"
            max_trailing_logs = 250000
            min_quorum = 3
            server_stabilization_time = "10s"
          }
        }
        seal "awskms" {
        }

        cluster_name = "vault_pri"
        log_level = "trace"
```

```
        max_lease_ttl = "768h"
        default_lease_ttl = "24h"
        service_registration "kubernetes" {}
        api_addr     = "https://HOSTNAME.vault-internal:8200"
        cluster_addr = "https://HOSTNAME.vault-internal:8201"
        ui = true
  # IRSA for KMS access
  serviceAccount:
    annotations:
      {
        eks.amazonaws.com/role-arn: "arn:aws:iam::617888641103:role/eksctl-vault-pri-kms-seal-role",
      }

  service:
    type: LoadBalancer
    externalTrafficPolicy: Local
    instanceSelector:
      enabled: false
    annotations:
      {
        service.beta.kubernetes.io/aws-load-balancer-type: external,
        service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing,
        service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
      }
  ingress:
    enabled: false

# Vault UI
ui:
  enabled: true
  serviceType: "LoadBalancer"
  publishNotReadyAddresses: false
  activeVaultPodOnly: true
# For Added Security, edit the below
#loadBalancerSourceRanges:
#   - < Your IP RANGE Ex. 10.0.0.0/16 >
#   - < YOUR SINGLE IP Ex. 1.78.23.3/32 >

```

Deploy Vault using Helm chart

```
$ helm install vault hashicorp/vault --namespace vault -f override-values.yaml --dry-run
$ helm install vault hashicorp/vault --namespace vault -f override-values.yaml
$ helm status vault -n vault
$ kubectl get pods -n vault -o wide
$ kubectl get service -n vault
```

Initialise Vault

```
$ kubectl -n vault exec vault-0 -it -- vault status
$ kubectl -n vault exec vault-0 -it -- vault operator init -recovery-shares=1 -recovery-threshold=1 -format json
```

> ⚠ This is for example only. It is not recommended for production to use only 1 recovery key. The recovery keys must be stored securely. Root keys should not used after initial configuration.

Verify Installation

```
$ export VAULT_ROOT_TOKEN=<root token>
$ for i in {0..2} ; do kubectl exec vault-$i -n vault -- vault status ; done
$ kubectl -n vault logs vault-0 | head -n 15
$ kubectl -n vault exec vault-0 -it -- sh -c "VAULT_TOKEN=$VAULT_ROOT_TOKEN vault operator members"
```

```
$ kubectl -n vault exec vault-0 -it -- sh -c "VAULT_TOKEN=$VAULT_ROOT_TOKEN vault operator raft list-peers"
$ kubectl -n vault exec vault-0 -it -- sh -c "VAULT_TOKEN=$VAULT_ROOT_TOKEN vault operator raft autopilot state"
```

Configure External DNS

Since the NLBs are internet facing custom external Domain Names can be configured.

Find the NLBs created for the services and get their DNS Names.

```
$ aws elbv2 describe-load-balancers --query 'LoadBalancers[?contains(LoadBalancerName, `vault`) == `true`]' --output json | jq -r '.[].DNSName'
$ kubectl get service vault-active -n vault -o jsonpath="{.status.loadBalancer.ingress[*].hostname}{'\n'}"
```

Add CNAME records in your DNS provider for the vault-active service NLB DNS Name.

`pri.prod.vault.<domain>` mapped to NLB DNS Name

Add another CNAME records in your DNS provider

`prod.vault.<domain>` mapped to `pri.prod.vault.<domain>`

You should be able to browse to `https://pri.prod.vault.<domain>:8200/` and `https://prod.vault<domain>:8200/` and Vault UI should open. You should be able to login with the root token.

Later when DR cluster is provisioned it will be easy to update the CNAME entry for `prod.vault.<domain>` to point to `dr.prod.vault.<domain>`

> The server certificate must have the CNAME records added as SAN when the certificate is issued.

Enable Auditing to Stdout

It is not possible to install the `logrotate` tool in the Vault pods. This means there is no way to stop the audit logs filling the auditStorage volume eventually and blocking the file audit device.

Enable logs on `stdout`. The logs can be streamed to a central log collector like DataDog or CloudWatch via FluentBit. See Set up Fluent Bit as a DaemonSet to send logs to CloudWatch Logs - Amazon CloudWatch

```
$ vault audit enable file file_path=stdout
```

> We recommend configuring multiple audit devices as a blocked audit device will cause Vault to stop responding. It is possible to configure a second audit device with logs going to `/dev/null`. If for any reason logs cannot be written to `stdout`

Vault will still continue to function albeit with loss of audit logs going to `/dev/null`.This is a trade-off between availability and compliance that should be carefully considered.

```
$ vault audit enable -path=audit-null file file_path=/dev/null
```

The logs to `stdout` are written to the host file system by default to a path `/var/log/containers/<container_id>`. It may be possible to use `logrotate` to rotate the logs on host file system, but we don't have guidance on how to do this.

TBD: Needs more investigation. See Logging Architecture

Vault EKS Multi-Cluster Architecture and Replication

Vault Multi-Cluster Architecture and Replication

Error loading the extension!

A cluster is the basic unit of Vault Enterprise replication which follows the leader-follower model. A leader cluster is referred to as the **primary** cluster and is considered the *system of record*. Data is streamed from the primary cluster to all **secondary** (follower) clusters.

Vault DR Secondary cluster deployment

The sections above for Vault on EKS Architecture and Vault on EKS Deployment can be repeated to also deploy a DR secondary cluster. The following values should be changed as applicable for the DR cluster.

- AWS Region
- AWS Account
- VPC CIDR
- Search "pri" and replace with "dr" wherever applicable in commands and configurations.

Once the DR cluster has been provisioned, ensure that it is initialised and it is running properly. Add this point we have two independent clusters, and the next step would be to connect them, so that this becomes a DR secondary cluster.

Vault Replication architecture on EKS

Vault on Kubernetes DR Replication Setup

Vault on Kubernetes DR Replication Setup

Enable replication on the primary cluster (Cluster A)

```
$ export VAULT_ADDR="https://pri.prod.vault.<domain>:8200" VAULT_TOKEN=<pri cluster root token>
$ vault read -format=json sys/replication/status
$ vault write -f sys/replication/dr/primary/enable primary_cluster_addr="https://pri.prod.vault.<domain>:8201"
$ $ vault read -address=https://pri.prod.vault.<domain>:8200 sys/replication/dr/status
```

Create a token

```
export DR_SECONDARY_TOKEN="$(vault write -field wrapping_token \
    sys/replication/dr/primary/secondary-token id="dr-cluster-$(date +%Y%m%d)" ttl=24h)"
```

Create a secret to test replication

```
$ vault secrets enable -path=secret kv-v2
$ vault kv put -mount=secret test hello=world
$ vault kv get -mount=secret test
```

Enable DR replication on the secondary cluster (Cluster B)

```
$ export VAULT_ADDR="https://dr.prod.vault.<domain>:8200" VAULT_TOKEN=<dr cluster root token>
$ vault write sys/replication/dr/secondary/enable \
    token=$DR_SECONDARY_TOKEN \
    primary_api_addr="https://pri.prod.vault.<domain>:8200" \
    ca_file="/vault/userconfig/vault-ca-crt/ca.crt"
$ vault read -address=https://dr.prod.vault.<domain>:8200 sys/replication/dr/status
```

DR Failover

Create a Batch DR operation token on Cluster-A

```
$ export VAULT_ADDR="https://pri.prod.vault.<domain>:8200" VAULT_TOKEN=<pri cluster root token>
```

Create Vault DR policy

```
vault policy write \
    dr-secondary-promotion - <<EOF
path "sys/replication/dr/secondary/promote" {
  capabilities = [ "update" ]
}

# To update the primary to connect
path "sys/replication/dr/secondary/update-primary" {
    capabilities = [ "update" ]
}

# Only if using integrated storage (raft) as the storage backend
# To read the current autopilot status
path "sys/storage/raft/autopilot/state" {
    capabilities = [ "update" , "read" ]
}
EOF
```

Create a token role

```
vault write auth/token/roles/failover-handler \
    allowed_policies=dr-secondary-promotion \
    orphan=true \
    renewable=false \
    token_type=batch
```

Create a token for the role

```
$ export CLUSTER_B_DR_OP_TOKEN=$(vault token create \
    -field=token -role=failover-handler -ttl=24h)
```

Securely store this batch token.

Simulate Failure of Primary Cluster

```
$ eksctl scale nodegroup --config-file vault-pri-cluster.yaml --nodes=0 [ --nodes-min=0 ] [ --nodes-max=6 ]
```

Change DNS entry to point prod.vault.<domain> to dr.prod.<domain> to route all traffic to DR cluster

Promote Secondary Cluster (Cluster B) to primary status

```
VAULT_ADDR=https://dr.prod.vault.<domain>:8220 \
    VAULT_TOKEN=$CLUSTER_B_ROOT_TOKEN \
    vault write -f sys/replication/dr/secondary/promote \
    dr_operation_token=$CLUSTER_B_DR_OP_TOKEN
```

Test access to Cluster B and replication of secrets.

```
$ export VAULT_ADDR="https://prod.vault.<domain>:8200" VAULT_TOKEN=<pri cluster root token>
$ vault kv list secret
$ vault kv get -mount=secret test
```

> After DR Secondary cluster is promoted to Primary the root token from the Primary has to be used. This is because all tokens, including the root token is replicated from primary to DR secondary cluster.

Demote Cluster A to secondary status

```
VAULT_ADDR=https://pri.prod.vault.platformrengineering.au:8200
vault write -f sys/replication/dr/primary/demote
```

Check that Cluster A is non-operational

```
vault kv get -mount=secret test
Error making API request.

URL: GET https://pri.prod.vault.platformrengineering.au:8200/v1/sys/internal/ui/mounts/secret
Code: 400. Errors:

* path disabled in replication DR secondary mode
```

Create a new secret on Cluster B

```
$ export VAULT_ADDR="https://prod.vault.<domain>:8200" VAULT_TOKEN=<pri cluster root token>
$ vault kv put -mount=secret test2 hello=world
```

Point demoted Cluster A to new Primary Cluster B

Generate a new secondary token

```
export CLUSTER_A_DR_SECONDARY_TOKEN="$(vault write -field wrapping_token \
    sys/replication/dr/primary/secondary-token id="pri-cluster-$(date +%Y%m%d)" ttl=24h)"

echo $CLUSTER_A_DR_SECONDARY_TOKEN
```

Point cluster B to cluster A

Update primary on Cluster A

```
export VAULT_ADDR=https://pri.prod.vault.platformrengineering.au:8200
vault write sys/replication/dr/secondary/update-primary \
    dr_operation_token=$CLUSTER_B_DR_OP_TOKEN \
    token=$CLUSTER_A_DR_SECONDARY_TOKEN \
    primary_api_addr="https://dr.prod.vault.platformrengineering.au:8200" \
    ca_file="/vault/userconfig/vault-ca-crt/ca.crt"

vault read --format=json sys/replication/status
```

Known Issues

- This deployment creates a EKS cluster with public subnet and vault services are exposed though an internet facing NLB. We do not recommend Vault to be exposed externally

ToDo

- [ ] AWS: Use IAM for service accounts instead on IAM on nodes.
- [ ] AWS: Use private endpoint for KMS
- [ ] AWS: Use internal LB with Route53 private hosted zones
- [ ] AWS: Use Terraform to create the entire infrastructure

Resources

AWS

Vault installation to Amazon Elastic Kubernetes Service via Helm | Vault | HashiCorp Developer

https://aws.amazon.com/blogs/containers/diving-into-iam-roles-for-service-accounts/

What is Amazon EKS? - Amazon EKS

Best Practices for Networking - Amazon EKS

Amazon VPC CNI - Amazon EKS

Learn about VPC CNI modes and configuration - Amazon EKS